

Structured Query Language (SQL) with MySQL

IEM 4723 Information Systems Design

Dr. Paritosh Ramanan

School of Industrial Engineering and Management
Oklahoma State University

Fall 2026

Slides courtesy Dr. Manjunath Kamath, IEM, OSU.

- 1 Introduction to SQL
- 2 Working with MySQL through the GUI
- 3 Using MySQL from Python

Introduction to SQL

What is SQL?

- SQL, or Structured Query Language, lets users pull the data they need out of a relational database, which can then be fed into analytical tools for further analysis.
- Its primary functions include creating tables, inserting data, updating existing data, and processing user queries.
- SQL is a user-friendly language, and is widely used in industrial engineering, operations research, and business applications.

Selecting a Column

- SQL statements are easy to understand. For example, `SELECT student_name FROM Class;` tells MySQL to select the `student_name` column from the `Class` table and display it.
- `SELECT` and `FROM` make up the core of a SQL statement for retrieving data.

```
SELECT column FROM table;
```

- Adding `DISTINCT` eliminates duplicates from the result.

```
SELECT DISTINCT column FROM table;
```

Selecting Multiple Columns

- More than one column can be selected at once.

```
SELECT column1, column2 FROM table;
```

- Using an asterisk instead of column names, as in `SELECT * FROM table;`, selects every column in the table.

The WHERE Clause

- The WHERE clause is used to select data given a condition, tailoring the result to what is actually needed.

```
SELECT Student_name, Student_Email FROM Class  
WHERE Student_CWID = '11122233';
```

- This query displays the name and email only for the student with the given CWID. Many different kinds of conditions can be used to filter data with WHERE.

Sorting with ORDER BY

- The ORDER BY clause displays data sorted in ascending or descending order.

```
SELECT * FROM customer_order ORDER BY Order_quantity  
ASC;
```

- This query displays every column from the customer_order table, sorted by Order_quantity in ascending order. Using DESC instead of ASC sorts in descending order.

Creating Tables

```
CREATE TABLE customer (  
    Customer_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Customer_name VARCHAR(20)  
);
```

- This query creates a table named `customer` with two columns, `Customer_ID` and `Customer_name`.
- `Customer_ID` is the primary key of the table, and is an integer. `Customer_name` is a variable character field that can hold up to 20 characters.

Inserting Values

```
INSERT INTO customer (Customer_name)  
VALUES ('John'), ('David');
```

- This query inserts John and David under the Customer_name column.
- No value was inserted for Customer_ID, since it was declared AUTO_INCREMENT and MySQL assigns it automatically.

Altering Tables

- The ALTER TABLE statement adds, deletes, or modifies columns in an existing table at any point.

```
ALTER TABLE customer  
ADD customer_address VARCHAR(225);
```

```
ALTER TABLE customer  
MODIFY customer_number BIGINT;
```

SQL Constraints

- SQL constraints specify rules for the data allowed in a table.
- NOT NULL ensures a column cannot have a NULL value, and UNIQUE ensures all values in a column are different.
- PRIMARY KEY combines NOT NULL and UNIQUE to uniquely identify each row, while FOREIGN KEY uniquely identifies a row in another table.
- CHECK ensures every value in a column satisfies a specific condition, DEFAULT sets a default value when none is specified, and INDEX is used to create and retrieve data very quickly.

Constraint Syntax

```
CREATE TABLE customer (  
    Customer_ID INT AUTO_INCREMENT PRIMARY KEY,  
    customer_name VARCHAR(225),  
    Customer_address VARCHAR(225) NOT NULL  
);
```

```
CREATE TABLE order (  
    Order_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Order_date DATE,  
    Customer_ID INT,  
    Order_amount DECIMAL(10, 2),  
    FOREIGN KEY (Customer_ID) REFERENCES customer(  
        Customer_ID)  
);
```

Aggregate Functions

- SUM returns the total of a numeric column, and AVG returns its average.
- COUNT returns the number of rows matching a given criteria.
- MAX and MIN return the largest and smallest values in the selected column.

Function	Syntax
MIN()	SELECT MIN(column_name) FROM table_name;
MAX()	SELECT MAX(column_name) FROM table_name;
COUNT()	SELECT COUNT(column_name) FROM table_name;
AVG()	SELECT AVG(column_name) FROM table_name;
SUM()	SELECT SUM(column_name) FROM table_name;

Working with MySQL through the GUI

Setting Up the Stack

- Tutorials are located in:
https://github.com/disys-lab/docker_tutorial.
- Go ahead and pull the latest updates using Github App.
- Navigate to the directory inside `docker_tutorial/mysql-dev` using the Terminal/CommandPrompt (Similar to Assignment 1)
- Once youre inside execute this command:
`docker compose up -d --force-recreate`.
- Running `docker compose up -d --force-recreate` builds and starts all three containers together.

Setting Up the Stack

```
version: '3.8'

services:
  mysql:
    image: mysql:latest
    container_name: mysql-container
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: OSUGoPokes
      MYSQL_DATABASE: iem4723_mysql_tutorial
    ports:
      - "3306:3306"
    volumes:
      - mysql-data:/var/lib/mysql

  phpmyadmin:
    image: phpmyadmin:latest
    container_name: phpmyadmin-container
    restart: always
    ports:
      - "8081:80"
    environment:
      PMA_HOST: mysql
      PMA_USER: root
      PMA_PASSWORD: OSUGoPokes
    depends_on:
      - mysql

volumes:
  mysql-data:
```

```
docker compose up -d --force-recreate
```

- The docker-compose.yml defines three services, a mysql container, a phpmyadmin container for browsing the database through a web GUI, and a jupyter container for working with the database from Python.
- Running `docker compose up -d --force-recreate` builds and starts all three containers together.

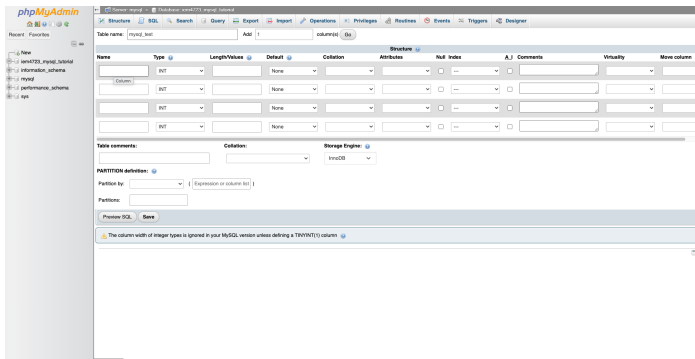
If a Container Doesn't Come Up

- Sometimes one of the three containers, mysql, phpmyadmin, or jupyter, does not start along with the others.
- You can bring up just that one service by name, using the same service names from `docker-compose.yml`.

```
docker compose up mysql -d  
docker compose up phpmyadmin -d  
docker compose up jupyter -d
```

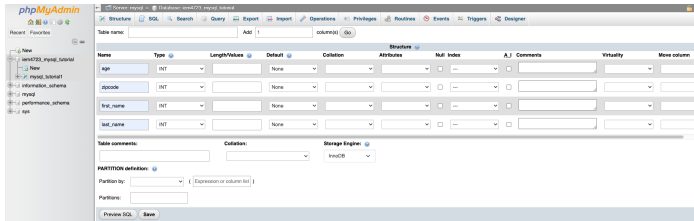
- Replace the service name with whichever one is missing. Run `docker ps` afterward to confirm all three are up and running.

Creating a Table



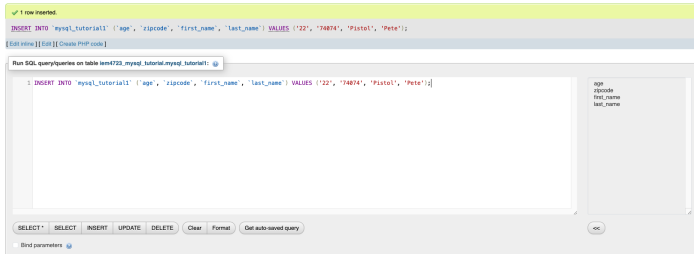
- phpMyAdmin's Structure tab lets you create a table by hand, entering a name for each column, its data type, and any other properties, directly in the browser.

Table Structure



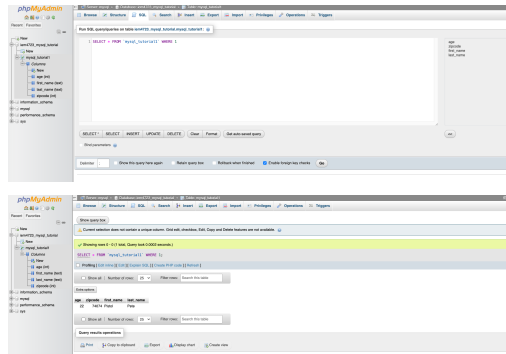
- Once a table is created, its Structure tab lists every column along with its type, so the schema can be reviewed or edited at any time.

Inserting Data via SQL



- phpMyAdmin's SQL tab runs any SQL statement directly against the database, such as an `INSERT INTO ... VALUES` statement, and confirms the result once it runs.

Querying a Table



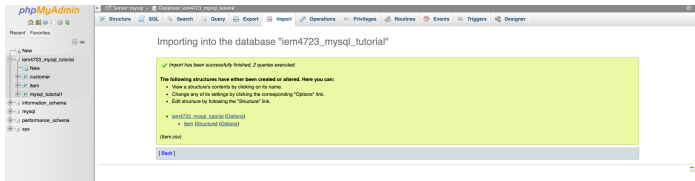
- Running a `SELECT * FROM ... WHERE 1` query in the SQL tab returns every row in the table, displayed in a results grid below the query box.

Importing a CSV File



- phpMyAdmin's Import tab can load a CSV file directly into the database. You choose the file, its character set, and confirm how columns are separated, enclosed, and escaped.
- If the first line of the file contains the column names, phpMyAdmin can use them as the table's column names automatically.

Import Confirmation



- Once the import finishes, phpMyAdmin reports the number of queries executed, and creates the new table, here item, directly from the CSV headers and rows.

Filtering with WHERE



Item_ID	Item_Description	Item_Price	Item_Weight	Item_AnnualSales	Item_StockLocation
121312-0026	Slacks	13	0	60	J6699
121332-0036	Slacks	10	0	2704	J6699
14	Shirts	6	0	96	A9309
18	Shirts	7	0	140	A9309
25	Shirts	14	0	1016	A9309
38	Shirts	15	0	52	A9309
699933	Slacks	12	0	506	L3234
71	Shirts	15	0	6672	A9309
841188	Slacks	15	0	85	L3234
842722	Slacks	14	0	131	L3234
PL530-JD	Footwear	4	4	748	A1976
PL638-JD	Footwear	6	4	306	A1976
PP24006	Neckwear	3	0	2	E4347
PP24007	Neckwear	13	0	24	E4347
TH3H390	Outerwear	7	4	3	D9463
TH4H580	Outerwear	7	4	1	D9463
TH4H690	Outerwear	14	4	13	D9463

☐ Show all | Number of rows: 25 | Filter rows: Search this table

- Adding a WHERE clause, such as `Item_Price >= 3`, filters the results down to only the rows that satisfy the condition.

Combining Conditions with AND

```
1 SELECT * FROM `item` WHERE Item_Price>=3 AND Item_Description="Slacks"
```

Item_ID	Item_Description	Item_Price	Item_Weight	Item_AnnualSales	Item_StockLocation
121312-0026	Slacks	13	0	60	J6699
121332-0036	Slacks	10	0	2704	J6699
699933	Slacks	12	0	506	L3234
841188	Slacks	15	0	85	L3234
842722	Slacks	14	0	131	L3234

- Multiple conditions can be combined with AND, such as filtering for items priced at three dollars or more that are also described as Slacks.

Using MySQL from Python

MySQL from Python

- The jupyter service in the same `docker-compose.yml` runs a Jupyter Notebook server with `pymysql` already installed, and mounts the project folder so notebooks can read local data files directly.
- Rather than a heavier library like `pandas`, this course uses plain `pymysql` cursors and Python's built-in `csv` module, so the Python version stays close to the raw SQL already covered.
- This gives the same access to the database as `phpMyAdmin`, but from Python code instead of the browser, which is useful for loading larger datasets or automating repeated tasks.

Connecting to MySQL

```
import pymysql

conn = pymysql.connect(
    host="mysql",
    port=3306,
    user="root",
    password="OSUGoPokes",
    database="iem4723_mysql_tutorial",
)
cursor = conn.cursor()
```

- `conn` is the live connection to the database, and `cursor` is what actually sends SQL statements through it and reads back whatever comes back.

Creating a Table in Python

Creating a Table

How did we create a table in phpMyAdmin again? We typed in column names and types by hand on the Structure tab. So how do we do the same thing in Python? We just write out that same CREATE TABLE statement as a string, and run it through *cursor*, same as always.

```
In [8]: create_vendor_manual_query = """
DROP TABLE IF EXISTS vendor_manual;
CREATE TABLE vendor_manual (
    Vendor_Num VARCHAR(10) PRIMARY KEY,
    Vendor_Name VARCHAR(50),
    Vendor_City VARCHAR(50),
    Vendor_State VARCHAR(2)
);
"""

for statement in create_vendor_manual_query.strip().split(";"):
    if statement.strip():
        cursor.execute(statement)

conn.commit()
print("vendor_manual table created.")

vendor_manual table created.
```

- The same CREATE TABLE statement from phpMyAdmin's Structure tab, written as a string and run through *cursor* instead.

Inserting Data via SQL in Python

Inserting Data via SQL

Remember the INSERT INTO ... VALUES statement we ran directly in phpMyAdmin's SQL tab? Same idea here. We write out the exact same statement as a plain string and run it through *cursor*, the same way we just ran CREATE TABLE above.

```
In [9]: insert_vendor_query = """
INSERT INTO vendor_manual (Vendor_Num, Vendor_Name, Vendor_City, Vendor_State)
VALUES ('V1107', 'CALATLANTIC GROUP', 'ARLINGTON', 'VA');
"""

cursor.execute(insert_vendor_query)
conn.commit()

cursor.execute("SELECT * FROM vendor_manual;")
for row in cursor.fetchall():
    print(row)

('V1107', 'CALATLANTIC GROUP', 'ARLINGTON', 'VA')
```

- The same INSERT INTO ... VALUES statement from the SQL tab, run and verified from Python.

Reading a CSV File in Python

Importing a CSV File

How did phpMyAdmin's Import tab build a table for us without us writing any SQL? It just read the CSV's header row and figured out the columns on its own. We don't get that shortcut in plain Python, so before we create anything or insert anything, we first need to read 'Vendor.csv' into Python, the exact same way we read 'Customer.csv' earlier.

```
In [10]: with open("../data/orders/Vendor.csv", newline='') as f:
        reader = csv.DictReader(f)
        vendor_rows = list(reader)

        vendor_rows[:3]

Out[10]: [{'Vendor_Num': 'V1107',
          'Vendor_Name': 'CALATLANTIC GROUP',
          'Vendor_Address': '1100 WILSON BOULEVARD',
          'Vendor_Phone': '8433481214',
          'Vendor_Email': 'CCKKPC@CALA.com',
          'Vendor_City': 'ARLINGTON',
          'Vendor_State': 'VA',
          'Vendor_ZIP': '22209'},
         {'Vendor_Num': 'V1119',
          'Vendor_Name': 'UNION SUPPLIERS',
          'Vendor_Address': '1400 DOUGLAS STREET',
          'Vendor_Phone': '7803978503',
          'Vendor_Email': 'VGB@UNIO.com',
          'Vendor_City': 'OMAHA',
          'Vendor_State': 'NE',
          'Vendor_ZIP': '68179'},
         {'Vendor_Num': 'V1320',
          'Vendor_Name': 'BUILDERS FIRSTSOURCE',
          'Vendor_Address': '2001 BRYAN STREET',
          'Vendor_Phone': '2857794738',
          'Vendor_Email': 'RDXH@BUIL.com',
          'Vendor_City': 'DALLAS',
          'Vendor_State': 'TX',
          'Vendor_ZIP': '75201'}]}
```

- Unlike phpMyAdmin's Import tab, plain Python has no built-in way to build a table straight from a CSV's header row, so the file is read first, using the same `csv.DictReader` pattern shown earlier for `Customer.csv`.

Creating the Table and Inserting the Data

Now that `vendor_rows` holds every row from the file, we create the `vendor` table to match its columns, then loop through `vendor_rows` and insert one row at a time, the exact same pattern we `customer`.

```
In [11]: create_vendor_table_query = """
DROP TABLE IF EXISTS vendor;
CREATE TABLE vendor (
    Vendor_Num VARCHAR(10) PRIMARY KEY,
    Vendor_Name VARCHAR(50),
    Vendor_Address VARCHAR(100),
    Vendor_Phone VARCHAR(15),
    Vendor_Email VARCHAR(100),
    Vendor_City VARCHAR(50),
    Vendor_State VARCHAR(2),
    Vendor_ZIP VARCHAR(10)
);
"""

for statement in create_vendor_table_query.strip().split(";"):
    if statement.strip():
        cursor.execute(statement)

insert_vendor_row_query = """
INSERT INTO vendor (Vendor_Num, Vendor_Name, Vendor_Address, Vendor_Phone, Vendor_Email, Vendor_City, Vendor_State, Vendor_ZIP)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s);
"""

for row in vendor_rows:
    cursor.execute(insert_vendor_row_query, (
        row["Vendor_Num"],
        row["Vendor_Name"],
        row["Vendor_Address"],
        row["Vendor_Phone"],
        row["Vendor_Email"],
        row["Vendor_City"],
        row["Vendor_State"],
        row["Vendor_ZIP"],
    ))

conn.commit()
print(f"Imported {len(vendor_rows)} rows into the vendor table.")
```

Imported 10 rows into the vendor table.

Querying a Table in Python

Querying a Table

Remember running `SELECT * FROM ... WHERE 1` in phpMyAdmin's SQL tab? That just returned every row in the table. What's the Python equivalent? We run the exact same query through `cursor`, then call `cursor.fetchall()` to grab every row it found.

```
In [12]: cursor.execute("SELECT * FROM vendor;")
         for row in cursor.fetchall():
             print(row)

('V1107', 'CALATLANTIC GROUP', '1100 WILSON BOULEVARD', '8433481214', 'CUKP@CALA.com', 'ARLINGTON', 'VA', '22209')
('V1119', 'UNION SUPPLIERS', '1400 DOUGLAS STREET', '7803978503', 'VGBP@UNIO.com', 'OMAHA', 'NE', '68179')
('V1320', 'BUILDERS FIRSTSOURCE', '2001 BRYAN STREET', '2857794738', 'RUXH@BUIL.com', 'DALLAS', 'TX', '75201')
('V1321', 'KIMBERLY-KENT', '351 PHELPS DRIVE', '3987045942', 'LWST@KIMB.com', 'IRVING', 'TX', '75038')
('V1443', 'NEWMONT WEARS', '6363 SOUTH FIDDLERS GREEN CIRCLE', '9773425160', 'YYZH@NEWM.com', 'GREENWOOD VILLAGE', 'CO', '80111')
('V1444', 'CUMMINS', '500 JACKSON STREET', '2746853534', 'PPNB@CUMM.com', 'COLUMBUS', 'IN', '47201')
('V1445', 'WALLYMART', '702 SW 8TH STREET', '9122776520', 'GUPD@WALM.com', 'BENTONVILLE', 'AR', '72716')
('V1446', 'US FINE HOLDING', '9399 WEST HIGGINS ROAD', '2753971873', 'TDSO@USFH.com', 'ROSEMONT', 'IL', '60018')
('V1448', 'SOUTHWEST GROUP', '2702 LOVE FIELD DRIVE', '7932062925', 'SBIR@SOUT.com', 'DALLAS', 'TX', '75235')
('V1450', 'ENERGY OUTFITTERS', '8111 WESTCHESTER DRIVE', '8562981897', 'OZBB@ENER.com', 'DALLAS', 'TX', '75225')
```

- `cursor.execute` runs the query, and `cursor.fetchall()` returns every row as a plain list of tuples, printed one at a time.

Filtering with WHERE in Python

Filtering with WHERE

Remember adding Item_Price >= 3 in the SQL tab earlier to filter our results? Same idea here. We filter vendor down to only the vendors based in Texas.

```
In [13]: cursor.execute("SELECT * FROM vendor WHERE Vendor_State = 'TX';")
         for row in cursor.fetchall():
             print(row)

('V1320', 'BUILDERS FIRSTSOURCE', '2001 BRYAN STREET', '2857794738', 'RUXH@BUIL.com', 'DALLAS', 'TX', '75201')
('V1321', 'KIMBERLY-KENT', '351 PHELPS DRIVE', '3987045942', 'LWST@KIMB.com', 'IRVING', 'TX', '75038')
('V1448', 'SOUTHWEST GROUP', '2702 LOVE FIELD DRIVE', '7932062925', 'SBIR@SOUT.com', 'DALLAS', 'TX', '75235')
('V1450', 'ENERGY OUTFITTERS', '8111 WESTCHESTER DRIVE', '8562981897', 'OZBB@ENER.com', 'DALLAS', 'TX', '75225')
```

- Filtering vendor down to only vendors based in Texas, the same WHERE clause used earlier in phpMyAdmin.

Combining Conditions with AND in Python

Combining Conditions with AND

Remember filtering for items priced at three dollars or more that were also Slacks? Same trick again. We narrow our Texas vendors down even further to only the ones based in Dallas.

```
In [14]: cursor.execute("SELECT * FROM vendor WHERE Vendor_State = 'TX' AND Vendor_City = 'DALLAS';")
         for row in cursor.fetchall():
             print(row)

('V1320', 'BUILDERS FIRSTSOURCE', '2001 BRYAN STREET', '2857794738', 'RUXXH@BUIL.com', 'DALLAS', 'TX', '75201')
('V1448', 'SOUTHWEST GROUP', '2702 LOVE FIELD DRIVE', '7932062925', 'SBIR@SOUT.com', 'DALLAS', 'TX', '75235')
('V1450', 'ENERGY OUTFITTERS', '8111 WESTCHESTER DRIVE', '8562981897', 'OZBB@ENER.com', 'DALLAS', 'TX', '75225')
```

- Narrowing the Texas vendors further to only those based in Dallas, combining two conditions with AND just as before.